Guia Completo de Desenvolvimento de Software — Squad MOD-2

Projeto: ODS 2026/2 — Oficina de Desenvolvimento de Software

Squad: MOD-2 (Mídia, Métricas e Interfaces)

Componentes: S4 (Evidência de Mídia), S5 (Métricas e Séries Temporais), B6 (Kit de Interface)

Alvos: NVIDIA Jetson Orin Nano (8 GB RAM) | Barramento B3 | Aplicações A1–A8

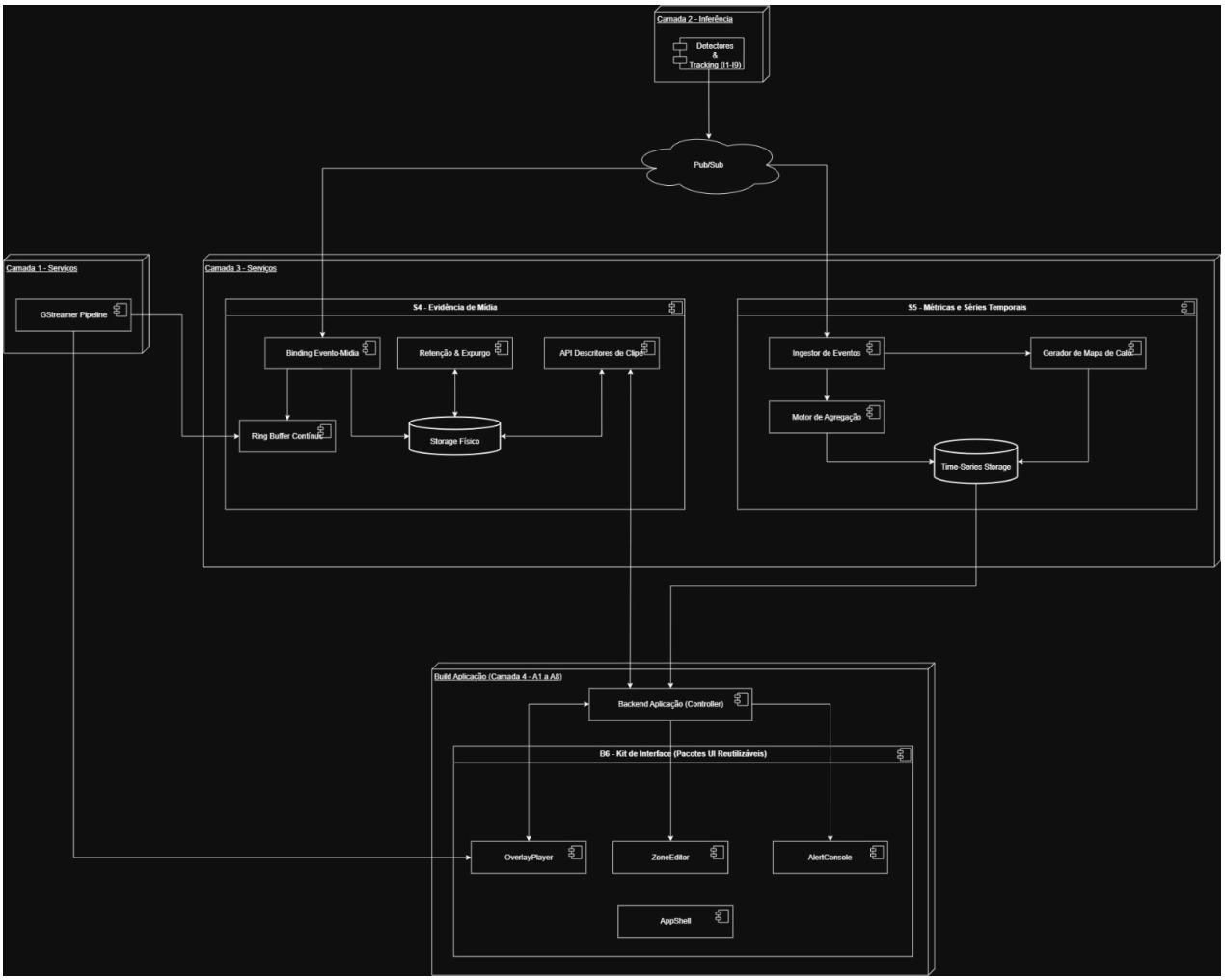
Diretrizes: Clean Code, Design Patterns (GoF), Domain-Driven Design (DDD) & Clean Architecture

---

1. Visão Geral e Responsabilidades do Squad MOD-2

O Squad **MOD-2** é responsável por três grandes pilares do ecossistema ODS 2026/2:

- **S4 (Evidência de Mídia — Camada 3):** Serviço responsável por gravar contínuamente em buffer de memória, salvar recortes de vídeo probatórios vinculados a eventos, controlar o armazenamento físico e garantir o expurgo automático por LGPD ou cota de disco.
- **S5 (Métricas e Séries Temporais — Camada 3):** Serviço responsável por transformar eventos brutos ruidosos em dados agregados no tempo (ocupação, permanência, horários de pico) e no espaço (mapas de calor em coordenadas de mundo via I3).
- **B6 (Kit de Interface e Operação — Camada 4):** Biblioteca de pacotes de interface reutilizáveis (`OverlayPlayer` , `ZoneEditor` , `AlertConsole` , `AppShell`) embutida diretamente na build de cada aplicação (A1–A8).



---

2. Quadro Executivo de Módulos (Entradas, Saídas, Arquitetura e Atribuições)

A tabela a seguir consolida as **funcionalidades**, **entradas esperadas**, **estruturas de dados de saída**, **arquitetura interna** e as **responsabilidades diretas dos membros do squad**:

Componente / Módulo	Funcionalidades Listadas	Entradas Esperadas (Inputs)	Saídas & Estrutura de Dados (Outputs)	Arquitetura Interna do Módulo	Responsável
S4.1 — Ring Buffer Contínuo	Captura contínua de vídeo em RAM; manutenção de	Stream H.264/RAW via GStreamer (RAM compartilhada `/dev/shm`).	Buffer circular em memória com ponteiros de frames	Facade + Ring Buffer de RAM (POSIX Shared Memory C++/Python).	**Henrique Azevedo**

	janela circular pré-evento (Ns).		`(timestamp, pointer, length)`.				
S4.2 — Binding Evento-Mídia	Mapeamento evento-mídia; recorte pré/pós evento; gravação de MP4 e hash SHA256.	Envelope Evento B1/B3 (JSON) + solicitação de extração da janela temporal.	Arquivo `.mp4` no NVMe + Registro SQLite: `{clip_id, event_id, file_path, hash_sha256}`.	Domain Command Handler + Encoder NVENC / Pipeline GStreamer.	**Davi Gomes**		
S4.3 — Retenção & Expurgo LGPD	Expurgo automático após 7 dias (LGPD); expurgo emergencial FIFO a 85% do NVMe; trava de auditoria.	Estado de ocupação do NVMe (`statvfs`) + sinalizador `is_locked_for_audit` do SQLite.	Log de Expurgo + Liberação de Espaço no NVMe + Update SQLite (`is_retained = false`).	Daemon Worker + Strategy Pattern (`RetentionStrategy`).	**Henrique Azevedo**		
S4.4 — API Descritores de Clipe	Exposição de metadados de mídia via REST/gRPC; registro imutável de acesso RBAC.	Requisição HTTP `GET /api/v1/clips/{clip_id}` com `token_rbac` e `justificativa`.	DTO JSON: `{clip_id, file_uri, start_time, end_time, points_of_interest[]}` + Audit Log.	REST/gRPC Controller + Clean Architecture Presentation Layer.	**Davi Gomes**		
S5.1 — Ingestor de Eventos B3	Consumo PubSub B3; parsing do Envelope B1/B3; debounce/histerese para ruídos na borda.	Fila PubSub B3 com mensagens JSON contendo envelope B1/B3.	Stream de Eventos Normalizados no domínio `{event_type, origin, coords, timestamp}`.	Event Consumer + Filter Strategy (Domain Service).	**Rafael Castro**		
S5.2 — Motor de Agregação	Cálculo de tempo de permanência (*dwell time*), taxa de ocupação instantânea e horário de pico.	Stream de Eventos Normalizados + Janelas temporais móveis (1m, 15m, 1h).	DTO Métrica Acumulada `{zone_id, dwell_time_avg, occupancy_peak, total_count}`.	Time Window Engine + Aggregator Strategy.	**Rafael Castro**		
S5.3 — Gerador de Mapa de Calor	Projeção espacial sem distorção em coordenadas métricas de plano de mundo.	Coordenadas de pixels (u, v) + Matriz de Homografia I3 ($H \in \mathbb{R}^{3\times 3}$).	Matriz 2D de Densidade Espacial `{grid_id, cells: [[x, y, density_value]]}`.	Spatial Coordinate Converter + Grid Density Calculator.	**Rafael Castro**		
S5.4 — Time-Series Store (DuckDB)	Armazenamento analítico colunar OLAP; Worker de Downsampling (Rollup 1s ➡️ 1m ➡️ 1h).	Métricas e Grades de Calor + Cron Trigger de Rollup.	Tabelas DuckDB OLAP + Consultas SQL de séries temporais para a aplicação.	DuckDB Repository + Downsampling Rollup Worker.	**Rafael Castro**		
B6.1 — OverlayPlayer	Player de vídeo com renderização acelerada por GPU de bounding boxes e polígonos.	Stream RTSP/WebRTC direto da Camada 1 + Array de metadados `{boxes[], zones[]}`.	Elemento de UI WebGL/Canvas2D sobreposto ao player em 60 FPS.	Componente WebGL/Canvas2D + Accelerated Video Player.	**Eduardo Gomes**		
B6.2 — ZoneEditor	Editor gráfico interativo vetorial para criação, ajuste e serialização de polígonos de risco.	Imagem da câmera + Interação de clique/arraste do usuário na tela.	Objeto JSON de Zona Serializada `{zone_id, name, type, points: [{x, y}]}` $[0.0, 1.0]$.	Vector Canvas Component + Topological Polygon Validator.	**Eduardo Gomes**		
B6.3 — AlertConsole	Feed reativo de notificações operacionais com ações de reconhecer, silenciar e encerrar.	Feed de Alertas via WebSocket/State Controller.	Evento de Ação Operacional `{alert_id, action: "ACK"	"MUTE"	"CLOSE", user_id}`.	Componente de Lista Reativa + Store de Gerenciamento de Estado.	**Eduardo Gomes**		
B6.4 — AppShell	Layout base, navegação, status de conexão da placa Jetson e Design System Tokens.	Propriedades da aplicação + status de hardware da Jetson.	Casca de Layout da Aplicação + Tokens CSS/Design System (Dark Mode).	Shell Layout Component + Design System Token Provider.	**Eduardo Gomes**		

---

3. Critérios de Divisão e Atribuição de Responsabilidades

A separação dos membros do **Squad MOD-2** foi definida estritamente sob **quatro critérios técnicos de engenharia**:

3.1. Alinhamento por Bounded Context & Domínio (DDD)

- **S4 (Evidência de Mídia):** Davi Gomes (Arquitetura & Regras de Domínio) e Henrique Azevedo (Infraestrutura de Baixo Nível & Mídia).
- **S5 (Métricas e Séries Temporais):** Rafael Castro (Processamento Analítico, Geometria I3 & DuckDB).
- **B6 (Kit de Interface e Operação):** Eduardo Gomes (Desenvolvimento Frontend, Canvas/WebGL & Design System).

3.2. Especialização de Stack & Afinidade Técnica

- **Henrique Azevedo (Sistemas & Baixo Nível):** Alocação de memória RAM compartilhada `/dev/shm`, ponteiros de Ring Buffer do GStreamer, monitoramento `statvfs` e daemons de expurgo.
- **Davi Gomes (Arquitetura & Integração de Aplicação):** Liderança do squad, definição de contratos DTOs de API, integridade probatória (hash SHA256), regras de auditoria RBAC/LGPD e barramento de eventos.
- **Rafael Castro (Engenharia de Dados & Matemática Espacial):** Pipelines analíticos, séries temporais no DuckDB, agregações móveis e álgebra linear para homografia espacial I3 ($H \in \mathbb{R}^{3\times 3}$).
- **Eduardo Gomes (Engenharia de Frontend & Computação Gráfica Web):** Pacotes de UI reutilizáveis (`@ods/b6-*`), renderização acelerada em Canvas2D/WebGL e monorepo npm.

3.3. Equilíbrio de Carga e Dimensionamento ODS (24 Pontos)

As 13 atividades do Squad MOD-2 (24 pontos) foram divididas em parcelas equivalentes de **6 pontos para cada desenvolvedor**:

- **Davi Gomes:** 3 atividades de S4 = **6 pts**
- **Henrique Azevedo:** 2 atividades de S4 = **6 pts**
- **Rafael Castro:** 4 atividades de S5 = **6 pts**
- **Eduardo Gomes:** 4 atividades de B6 = **6 pts**

---

4. Diretrizes de Arquitetura e Submódulos dos Componentes

> [!NOTE]

> As diretrizes abaixo estabelecem as **responsabilidades de alto nível e os contratos funcionais** de cada submódulo. Os detalhes exatos de implementação interna, bibliotecas específicas e escolha de estruturas internas de código devem ser discutidos e alinhados pelos responsáveis diretos durante a fase de desenvolvimento.

4.1. Componente S4 — Evidência de Mídia (Camada 3 - Serviço)

🎯 Diretrizes Gerais do Componente

Garantir a captura eficiente de evidências visuais (vídeos/snapshots) e gerenciar seu ciclo de vida em borda, desde o buffer circular inicial até o expurgo seguro no disco NVMe.

🧩 Submódulos e Diretrizes de Implementação:

S4.1 — Ring Buffer Contínuo (Alocação em RAM)

- **Responsável:** Henrique Azevedo
- **Diretriz de Design:** Manter um buffer circular em memória RAM compartilhada (ex: `/dev/shm`) para armazenar os últimos N segundos pré-evento sem desgastar o SSD NVMe.
- **Tópicos para Discussão do Desenvolvedor:**
- Definir o tamanho ideal da janela pré-evento (ex: 15s a 30s) de acordo com o consumo de RAM por câmera.
- Avaliar entre implementar a fila circular via C/C++ vinculada com ctypes ou usar abstrações nativas do GStreamer (`appsink` / `shmdata`).

S4.2 — Binding Evento-Mídia (Vinculação Imutável)

- **Responsável:** Davi Gomes
- **Diretriz de Design:** Ao receber a notificação de um evento do Barramento B3, extrair o trecho de vídeo pré e pós-evento da RAM, consolidar em um arquivo definitivo `.mp4` (ou snapshot `.jpg`) e gerar uma assinatura de integridade (ex: hash SHA256).
- **Tópicos para Discussão do Desenvolvedor:**
- Definir se a codificação do vídeo utilizará aceleração gráfica via NVENC ou encoder padrão H.264.
- Alinhar com a equipe o formato final dos metadados gravados na tabela SQLite.

S4.3 — Retenção & Expurgo LGPD (Purge Manager Daemon)

- **Responsável:** Henrique Azevedo
- **Diretriz de Design:** Garantir a remoção automática de mídias por ultrapassagem de prazo da LGPD (ex: 7 dias) ou por estouro emergencial de cota de disco NVMe (ex: 85%), respeitando a trava de auditoria (`is_locked_for_audit`).
- **Tópicos para Discussão do Desenvolvedor:**
- Escolher a frequência da rotina de limpeza (ex: a cada 1h ou orientada a eventos de I/O).
- Alinhar a política FIFO de descarte para mídias não travadas em caso de emergência de disco.

S4.4 — API Descritores de Clipe

- **Responsável:** Davi Gomes
- **Diretriz de Design:** Expor metadados de mídias via API local (REST ou gRPC) para fornecimento de URIs e pontos de interesse para a aplicação, sem trafegar o binário pesado do vídeo no payload JSON.
- **Tópicos para Discussão do Desenvolvedor:**

- Definir a estrutura exata do contrato DTO e os campos de pontos de interesse (*POIs*).
- Implementar o registro de log de auditoria RBAC no momento da consulta para atendimento à LGPD.

4.2. Componente S5 — Métricas e Séries Temporais (Camada 3 - Serviço)

🎯 Diretrizes Gerais do Componente

Transformar eventos ruidosos em séries temporais agregadas de ocupação, tempo de permanência (*dwell time*), picos de movimento e mapas de calor espacial.

🧩 Submódulos e Diretrizes de Implementação:

S5.1 — Ingestor de Eventos B3

- **Responsável:** Rafael Castro
- **Diretriz de Design:** Consumir eventos do Barramento B3, validar o envelope B1/B3 e aplicar tratamento de histerese/debounce para ignorar ruídos na borda das zonas.
- **Tópicos para Discussão do Desenvolvedor:**
- Definir o parâmetro de tempo de debounce (ex: suprimir repetições em janelas menores que 2s).

S5.2 — Motor de Agregação (Time Window Engine)

- **Responsável:** Rafael Castro
- **Diretriz de Design:** Calcular a permanência média dos objetos nas zonas, contagens instantâneas/acumuladas e identificar horários de pico.
- **Tópicos para Discussão do Desenvolvedor:**
- Ajustar as janelas temporais de agregação móvel (ex: 1m, 15m, 1h).

S5.3 — Gerador de Mapa de Calor (Homografia I3)

- **Responsável:** Rafael Castro
- **Diretriz de Design:** Converter coordenadas relativas da imagem (u, v) para coordenadas métricas de mundo (X, Y) em metros usando a Matriz de Homografia fornecida pelo componente I3.
- **Tópicos para Discussão do Desenvolvedor:**
- Definir a resolução da grade bidimensional (*Spatial Density Grid*) em metros (ex: células de 0.5m x 0.5m).

S5.4 — Time-Series Store (DuckDB & Rollup Worker)

- **Responsável:** Rafael Castro
- **Diretriz de Design:** Persistir séries temporais em banco OLAP local (DuckDB) e executar rotinas de compactação progressiva (Rollup/Downsampling) para poupar disco.
- **Tópicos para Discussão do Desenvolvedor:**
- Definir o ciclo de rollup de dados históricos (ex: manter segundos durante 24h, minutos durante 7 dias e horas para prazos maiores).

4.3. Componente B6 — Kit de Interface e Operação (Camada 4 - Bibliotecas UI)

🎯 Diretrizes Gerais do Componente

Publicar pacotes de UI reutilizáveis (`@ods/b6-*`) em formato de biblioteca para serem incorporados diretamente nas builds das aplicações (A1–A8).

🧩 Submódulos e Diretrizes de Implementação:

B6.1 — OverlayPlayer (`@ods/b6-overlay-player`)

- **Responsável:** Eduardo Gomes
- **Diretriz de Design:** Renderizar stream de vídeo em tempo real com sobreposição vetorial de caixas delimitadoras e zonas sem degradar a performance da CPU.
- **Tópicos para Discussão do Desenvolvedor:**
- Avaliar a melhor estratégia gráfica de sobreposição (WebGL vs Canvas2D aceleração nativa).

B6.2 — ZoneEditor (`@ods/b6-zone-editor`)

- **Responsável:** Eduardo Gomes
- **Diretriz de Design:** Permitir que o usuário desenhe e edite polígonos de zonas na tela, serializando os vértices em coordenadas normalizadas $[0.0, 1.0]$.
- **Tópicos para Discussão do Desenvolvedor:**
- Definir os mecanismos de validação topológica (impedir polígonos cruzados).

B6.3 — AlertConsole (`@ods/b6-alert-console`)

- **Responsável:** Eduardo Gomes
- **Diretriz de Design:** Exibir feed reativo de alertas operacionais e capturar comandos de ação (**ACK**, **MUTE**, **CLOSE**).
- **Tópicos para Discussão do Desenvolvedor:**
- Estruturar o gerenciamento de estado da lista de alertas.

B6.4 — AppShell (`@ods/b6-app-shell`)

- **Responsável:** Eduardo Gomes
- **Diretriz de Design:** Fornecer a estrutura base de layout, navegação, indicadores de status da Jetson e tokens do Design System ODS.

- **Tópicos para Discussão do Desenvolvedor:**
- Definir a paleta de temas (Dark Mode) e tokens visuais compartilhados.

---

5. Arquitetura de Código, DDD e Estrutura de Pastas

Cada serviço backend (S4 e S5) deve ser construído seguindo rigorosamente a **Clean Architecture** (Arquitetura Limpa), dividida em 4 camadas internas sem dependências circulares:

```
src/
├── domain/          # 1. CORAÇÃO: Entidades, VOs, Contratos de Repositórios (Zero dependências externas)
├── application/     # 2. CASOS DE USO: Orquestração das regras de negócio e DTOs
├── infrastructure/  # 3. INFRAESTRUTURA: SQLite, DuckDB, GStreamer, Pub/Sub B3
└── presentation/    # 4. APIS: Controllers REST / gRPC
```

📂 Estrutura de Diretórios Recomendada

```
mod2/
├── s4-media-evidence/
│   ├── src/
│   │   ├── domain/
│   │   │   ├── entities/            # MediaClip.py
│   │   │   ├── value_objects/       # TimeWindow.py, ClipDescriptor.py
│   │   │   ├── repositories/        # IMediaClipRepository.py (Interface)
│   │   │   ├── strategies/          # RetentionStrategy.py (GoF Strategy)
│   │   │   └── exceptions/          # DomainException.py
│   │   ├── application/
│   │   │   ├── use_cases/           # ExtractClipUseCase.py, PurgeMediaUseCase.py
│   │   │   └── dtos/                # ClipDescriptorDTO.py
│   │   ├── infrastructure/
│   │   │   ├── database/            # SQLiteMediaClipRepository.py
│   │   │   ├── gstreamer/           # GStreamerRAMBufferAdapter.py
│   │   │   └── b3_bus/              # B3EventConsumer.py
│   │   └── presentation/
│   │       └── http/                # ClipDescriptorController.py
│   └── tests/
│       ├── unit/                    # Testes puros de domínio (sem I/O)
│       └── integration/             # Testes com SQLite e sistema de arquivos
│
├── s5-time-series/
│   ├── src/
│   │   ├── domain/
│   │   │   ├── entities/            # HeatmapGrid.py, OccupancyMetric.py
│   │   │   ├── value_objects/       # WorldCoordinate.py, SpatialCell.py
│   │   │   ├── repositories/        # ITimeSeriesRepository.py
│   │   │   └── services/            # HeatmapDomainService.py
│   │   ├── application/
│   │   │   ├── use_cases/           # AggregateDwellTimeUseCase.py, GenerateHeatmapUseCase.py
│   │   │   └── dtos/                # HeatmapQueryDTO.py
│   │   ├── infrastructure/
│   │   │   ├── database/            # DuckDBTimeSeriesRepository.py
│   │   │   └── i3_calibration/      # HomographyCalibratorAdapter.py
│   │   └── presentation/
│   │       └── http/                # MetricsQueryController.py
│   └── tests/
│       ├── unit/
│       └── integration/
│
└── b6-ui-kit/                       # Monorepo NPM
    ├── packages/
    │   ├── overlay-player/
    │   ├── zone-editor/
    │   ├── alert-console/
    │   └── app-shell/
    └── package.json
```

---

6. Padrões de Projeto (GoF) a Utilizar

Para manter o código manutenível e extensível, aplique os seguintes **Design Patterns**:

Problema de Engenharia	Padrão GoF Aplicado	Onde e Como Utilizar

Alternar regras de expurgo por tempo (LGPD) ou por espaço (Cota 85% NVMe)	**Strategy**	`RetentionStrategy` no S4. Permite trocar o algoritmo de deleção em tempo de execução.
Isolar o banco de dados (SQLite/DuckDB) das regras de negócio do domínio	**Repository**	Interfaces `IMediaClipRepository` e `ITimeSeriesRepository` no domínio.
Inscrição e reação aos eventos brutos do Barramento B3	**Observer / Consumer**	`B3EventConsumer` escuta a fila Pub/Sub e dispara os Casos de Uso.
Fornecer uma interface simplificada para extração de vídeo do RingBuffer	**Facade**	`RAMBufferFacade` esconde a complexidade de ponteiros de memória do GStreamer.

---

7. Regras de Código (Clean Code & Segurança)

7.1. Nomenclatura Expressiva

- **Váriaveis e Funções:** Devem revelar a intenção pura sem necessidade de comentários explicativos.
- ❌ `def proc(d, t):`
- ✅ `def extract_clip_for_event(event_id: str, time_window: TimeWindow) -> MediaClip:`
- **Booleanos:** Devem ser lidos como perguntas predicadas (`is_retained`, `has_exceeded_quota`, `is_locked_for_audit`).

7.2. Regra de Funções Pequenas

- Funções devem ter apenas **uma responsabilidade** (Single Responsibility Principle - SRP) e idealmente até 20 linhas.
- **Separação Comando-Consulta (CQS):** Uma função ou altera o estado do sistema (*Command*) ou retorna um valor (*Query*), nunca os dois simultaneamente.

7.3. Hierarquia Estrita de Exceções

Nunca lance exceções genéricas como `Exception` ou `RuntimeException`. Utilize exceções domain-specific:

```
class ODSBaseException(Exception):
    """Exceção base para todo o sistema ODS."""

class DomainError(ODSBaseException):
    """Erros de violação de regras de negócio do domínio."""

class InvalidTimeWindowError(DomainError):
    """Lançado quando start_time >= end_time."""

class DiskQuotaExceededError(DomainError):
    """Lançado quando o NVMe atinge limite sem mídia para expurgar."""

class ApplicationError(ODSBaseException):
    """Erros na camada de casos de uso."""

class MediaFileNotFoundError(ApplicationError):
    """Lançado quando o arquivo .mp4 não existe mais no NVMe."""
```

---

8. Estratégia de Testes (TDD & Qualidade)

> **"Código sem testes não está limpo." — Dave Thomas**

8.1. Regras de Testes de Unidade (`tests/unit/`)

- Testes de unidade testam **apenas a lógica pura do domínio**.
- **Zero I/O:** Sem chamadas de banco de dados real, sem rede, sem sistema de arquivos real (use *Stubs* e *Fakes* em memória).
- **Nomenclatura Obrigatória:**

$$\text{deve_[resultado_esperado]_quando_[condição_testada]}$$

Exemplo de Teste de Unidade Limpo:

```
# tests/unit/domain/test_time_window.py
import pytest
from datetime import datetime, timedelta
from src.domain.value_objects.time_window import TimeWindow
from src.domain.exceptions import InvalidTimeWindowError

def test_deve_calcular_duracao_corretamente_quando_janela_for_valida():
    start = datetime(2026, 9, 10, 10, 0, 0)
    end = start + timedelta(seconds=30)

    tw = TimeWindow(start_time=start, end_time=end)
```

```
    assert tw.duration_seconds == 30.0

def test_deve_lancar_excecao_quando_start_time_for_maior_ou_igual_ao_end_time():
    start = datetime(2026, 9, 10, 10, 0, 0)
    end = start - timedelta(seconds=1)

    with pytest.raises(InvalidTimeWindowError):
        TimeWindow(start_time=start, end_time=end)
```

---

9. Checklist Obrigatório para o Desenvolvedor (Antes do Commit)

Antes de submeter qualquer Pull Request (PR) ou código para revisão do squad MOD-2, garanta que:

- [ ] **Clean Architecture Respeitada:** As dependências apontam obrigatoriamente para dentro (Domínio não importa nada da Infraestrutura ou Frameworks).
- [ ] **Sem Vídeo no Barramento:** Nenhuma mídia binária ou arquivo pesado foi transmitido pelo barramento B3.
- [ ] **Nomes Limpos:** Variáveis e métodos dispensam comentários para explicar o que fazem.
- [ ] **Tratamento de Erros:** Todas as falhas lançam exceções tipadas de `DomainError` ou `ApplicationError`.
- [ ] **Testes de Unidade:** As regras de negócio alteradas possuem testes de unidade passando na suíte automatizada.
- [ ] **Conformidade LGPD:** As novas entidades de mídia incluem sinalizadores de tempo de vida e flag de auditoria.

Guia Completo de Desenvolvimento de Software — Squad MOD-2

Complemento: Planejamento da Sprint 3 — Integração e Armazenamento em Nuvem

> Complemento do guia docs/responsability.md / `docs/GUIA_DESENVOLVIMENTO_MOD2.pdf` (seções 1–9). Estende as **seções 4 (Diretrizes de Arquitetura e Submódulos)**, **5 (Arquitetura de Código, DDD)**, **6 (Padrões GoF)** e **9 (Checklist do Desenvolvedor)** para o ciclo de integração entre componentes e substituição do disco físico por armazenamento em nuvem. Referências externas: `docs/ODS 2026_2 – Arquitetura 6c.html` (Especificação de interfaces 1.0) e `docs/Guia_Kafka_B2 (1).PDF`.

---

10. Planejamento da Sprint 3

10.1 Objetivo e escopo

A Sprint 3 fecha a lacuna entre os componentes já implementados isoladamente (S4 em `ods-26-2/s4`, S5 em `ods-26-2/s5`, B6 em `ods-26-2/b6`) e a arquitetura 6c, em dois eixos:

1. **Integração dos componentes** entre si e com o **barramento B2/B3** (Kafka), conforme o Guia de Uso do Kafka.
2. **Armazenamento em nuvem (Google Cloud)** substituindo o disco físico para as mídias probatórias do S4, mantendo os metadados localmente.

Aplica-se a todo o Squad MOD-2 (Davi Gomes, Henrique Azevedo, Rafael Castro, Eduardo Gomes), na divisão de responsabilidades da seção 3 do guia.

10.2 Análise de aderência à arquitetura 6c

O que a 6c muda para o MOD-2:

- **S4 passa a consumir S2** e grava automaticamente todo `ods.s2.event` com `status` `CONFIRMED` / `UNCERTAIN` e `phase` `STARTED`, com a pré-rolagem configurada.
- S4 atende **`ods.s4.capture.command`** (reply imediato `{"status":"OK","clip_id":…}`; descritor completo por `ods.s4.clip`) e **`ods.s4.clip.query/.reply`** (por `clip_id` ou `event_id`; lista vazia se expirou).
- O descritor **`ods.s4.clip`** tem `media_type` (CLIP/SNAPSHOT) e **`expires_at`** **obrigatório**; `media_uri` é referência (ex.: `s4://clips/AAAA/MM/DD/<uuid>.mp4`), nunca o binário.
- S4 consome também **`ods.session.started`** (troca de sessão zera estado e `sequence`) e os streams `ods.p4.frame` / `ods.p5.audio_chunk` (metadados fora do barramento).
- **S5** consome `ods.s1.*`, `ods.s2.*` e I3; produz `ods.s5.zone_metrics` e `ods.s5.event_metrics`; atende `ods.s5.metrics.query/.reply` e `ods.s5.heatmap.query/.reply`.

- **B6** é camada 4 embutida nas aplicações; o editor de zonas alimenta o artefato **ZONES** (formato §8 da spec).

O que o Guia Kafka (B2) exige:

- Broker em Google Cloud `34.66.235.91:9093` é **temporário** — o endereço tem de ser **configurável** (flag/env), nunca fixo no código.
- Cliente por linguagem: C++ → **librdkafka**; Python → **confluent-kafka**; Node/TS → kafkajs.
- Validação do envelope B1 em três níveis (JSON válido → campos obrigatórios/schema/versão → payload); fora do formato: **descartar e registrar**.
- Idempotência por `message_id`; nenhum binário no payload (só referências); `sequence` para detecção de perda.
- **Créditos GCP**: sem laços sem condição de parada, poll ≥ 1 s, encerrar processos de teste, combinar carga com a squad TRA-1.

Estado atual (ponto de partida):

Componente	Estado
`ods-26-2/s4` (C++)	Completo: ring buffer real (GStreamer), binding evento-mídia, retenção/expurgo LGPD, REST (cpp-httplib), Docker multi-stage. `B3EventConsumer` é **só contrato**.
`ods-26-2/s5` (Python)	Clean Architecture com DuckDB/FastAPI; consumidor `InMemoryB3EventConsumer` e envelope antigo (`event_type/origin/coords/timestamp`); sem Kafka real.
`ods-26-2/b6` (TypeScript)	OverlayPlayer e ZoneEditor com testes Vitest; **mockados**, sem ligação com S4 nem export de artefato ZONES.

10.3 Decisões da sprint

Tema	Decisão
Escopo de integração	Todos os componentes: **S4 + S5 + B6 + barramento Kafka**
Nuvem	**Clipes/snapshots em GCS**; metadados continuam no **SQLite local**
Cliente GCS (C++)	**REST JSON API via cpp-httplib (vendado) + JWT service account (RS256/OpenSSL)**
Kafka	**Integração real ponta a ponta** (librdkafka), bootstrap configurável
URI do descritor	Mantém `s4://clips/...` canônica no DTO; `GcsFileStorage` mapeia para a chave do bucket; o consumidor recebe **URL assinada V4** de curta duração
Storage local	`--storage local` continua disponível (testes/offline); híbrido: grava local → upload assíncrono → publica `ods.s4.clip` só após sucesso

10.4 Trabalhos (WIs) e responsáveis

Em linha com a seção 3 do guia (bounded context + especialização de stack).

Frente A — S4 + Kafka (Davi, integração)

WI	Entregável
A1	**Envelope B1 em C++** (`infrastructure/b3_bus/envelope.{hpp,cpp}`): builder dos 13 campos (perfis EV-SERVIÇO, CMD, RPL), validador em 3 níveis, `sequence` por producer+session, dedup por `message_id` .
A2	`KafkaEventTransport` (implementa o contrato `B3EventConsumer` da seção 6 do guia — Observer/Consumer): librdkafka, bootstrap de `--kafka-brokers` /env, `group.id` próprio (`S4-…`), `auto.offset.reset=earliest` , poll ≥ 1 s, `stop()` no SIGTERM. Consome `ods.s2.event` , `ods.session.started` , `ods.s4.capture.command` , `ods.s4.clip.query` .
A3	**Consumo → casos de uso**: `ods.s2.event` (CONFIRMED/UNCERTAIN + STARTED) dispara `ExtractClipUseCase` com pré-rolagem (`ConsumeS2EventUseCase`); `capture.command` → reply imediato + descritor por `ods.s4.clip` ; `clip.query/.reply` sobre o repositório.
A4	**Produtor**: publicar `ods.s4.clip` (EV-SERVIÇO, `correlation_id` = event_id/comando) ao concluir a extração; replies RPL para command/query. **Risco**: naming de tópico de reply não consta no guia → alinhar com TRA-1 (padrão provisório `ods.s4.<nome>.reply`).
A5	`main.cpp` : flags `--kafka-brokers` , `--kafka-group-id` , `--storage local|gcs` , `--gcs-bucket` , `--gcs-credentials` ; transporte desabilitado se ausente (POST `/api/v1/events` continua funcionando).

Frente B — Nuvem (Davi e Henrique, substitui o disco físico)

WI	Entregável
B1	`GcsFileStorage : IFileStorage` (`infrastructure/gcs/`): `write/exists/sizeOf/remove` via JSON API (`uploadType=media` , `GET/DELETE /storage/v1/b/…/o/…`); OAuth2 com JWT RS256 (OpenSSL) → `oauth2.googleapis.com/token` , com **cache e refresh do token**; bucket/key derivados de `media_uri` .
B2	**URL assinada V4** (`GcsUrlSigner`): assinatura RSA do canonical request → HTTPS temporário (TTL ~15 min) para B6/app reproduzir o clipe.
B3	**Upload resumable** (`uploadType=resumable`) para arquivos ≥ 5 MB — evita estouro de limite e falha parcial.
B4	**Retenção/expurgo na nuvem — responsável: Henrique** (evolução do seu S4.3): `StatvfsDiskUsageProvider` não vale para bucket → `IBucketUsageProvider` (JSON API `GET /storage/v1/b/<bucket>` → `storage.bytesUsed`) para a cota FIFO; `PurgeMediaUseCase` remove do GCS via `IFileStorage` ; **lifecycle rule** no bucket como rede de segurança (apagar após `expires_at`); `expires_at` = retenção (7 dias LGPD).
B5	**Migração/fallback**: `--storage local` mantido; híbrido grava local → upload assíncrono → só publica `ods.s4.clip` após sucesso (retry limitado).

Frente C — S5 integrado (Rafael, `ods-26-2/s5`)

WI	Entregável
C1	**Adaptar ao envelope B1/6c**: parsing dos 13 campos; assinar `ods.s1.zone_transition` , `ods.s1.zone_occupancy` , `ods.s2.event` (hoje espera `event_type/origin/coords/timestamp`). Aceitar os dois formatos temporariamente para não quebrar os 59 testes.
C2	`KafkaB3EventConsumer` **(confluent-kafka)** substituindo o `InMemory` (mantido nos testes): grupo `S5-…` , bootstrap configurável, validação/descarte com log, idempotência.
C3	**Producer/reply**: publicar `ods.s5.zone_metrics` e `ods.s5.event_metrics` ; atender `ods.s5.metrics.query/.reply` e `ods.s5.heatmap.query/.reply` .

WI	Entregável
C4	**Artefatos**: carregar ZONES/CALIBRATION na partida (formato §8 da spec) em vez de arquivo solto de homografia.

Frente D — B6 integrado (Eduardo, `ods-26-2/b6`)

WI	Entregável
D1	**OverlayPlayer ↔ S4**: cliente `S4ClipClient` (REST: `GET /api/v1/clips/{id}`, `POST /api/v1/events`) obtendo descritor + URL assinada, reproduzindo e sincronizando `points_of_interest` na timeline.
D2	**ZoneEditor → artefato ZONES**: export no formato §8 (`space_id`, `coordinates`, `source_ids`, `zones[].polygon/enter_hysteresis_ms/exit_hysteresis_ms/max_occupancy/classes`), pronto para S1/S5.
D3	**Demos reais**: páginas `index.html` apontando para o S4 rodando (Vite + proxy), sem mocks.

10.5 Cronograma (2 semanas)

Fase	Dias	Entregável
1	1–2	WI-A1 envelope + testes unit · WI-B1 `GcsFileStorage` + fakes · WI-C1
2	3–5	WI-A2/A3 transporte + consumo · WI-B2/B4 URL assinada + expurgo GCS · WI-C2
3	6–8	WI-A4/A5 produtor + replies · WI-B5 híbrido/upload · WI-C3 · WI-D1
4	9–10	WI-D2/D3 · **E2E do squad** · docs + Docker

10.6 Verificação (Definition of Done)

Complementa a seção 8 (Estratégia de Testes) e a seção 9 (Checklist) do guia.

```
# S4 (C++)
cmake -S mod2/s4-media-evidence -B build && cmake --build build
ctest --test-dir build --output-on-failure

# S5 (Python)
pytest

# B6 (TypeScript)
npx vitest
```

Smoke E2E do squad (script novo `scripts/sprint3_smoke.sh`):

1. Publicar `ods.s2.event` (CONFIRMED/STARTED) no broker do guia → S4 extrai o clipe, sobe no bucket e publica `ods.s4.clip`.
2. `GET /api/v1/clips/{id}` → `media_uri` + `expires_at` + hash; URL assinada baixa o MP4 com SHA-256 idêntico.
3. `ods.s4.capture.command` (SNAPSHOT) → reply OK + descritor; `ods.s4.clip.query` por `event_id` → lista.
4. S5 consome os mesmos eventos → `GET /api/v5/metrics/{zone_id}` com dados reais.
5. B6 OverlayPlayer reproduz o clipe pela URL assinada e exibe POIs; ZoneEditor exporta ZONES válido.

6. Expurgo: `expires_at` vencido → objeto removido do bucket + `is_retained=false` no SQLite.

10.7 Riscos e cuidados

- **Créditos GCP**: testes contra o broker real curtos e supervisionados, poll ≥ 1 s, encerrar processos (guia §4.5); scripts de carga combinados com TRA-1.
- **Reply de command/query**: naming de tópico não definido no guia → alinhar com TRA-1 (dono de B1/B3).
- **Credenciais**: service account fora do repositório (`.gitignore`), sem logar tokens; rotação combinada com a equipe.
- **librdkafka no Docker/Jetson**: `librdkafka-dev` no builder e `librdkafka1` no runtime (amd64+arm64 no apt); flag `ODS_S4_WITH_KAFKA=ON/OFF` , como a do GStreamer.
- **LGPD na nuvem**: regra dos 7 dias não muda — `expires_at` + expurgo removendo do bucket; bucket autenticado (sem acesso público) e URL assinada curta.
- **Fuso/hora**: tudo ISO 8601 UTC com milissegundos; `sequence` / `session_id` zerados em `ods.session.started` .
- **Quebra de contrato do S5**: aceitar envelope antigo e 6c temporariamente (dual-mode) para não travar a suíte existente.

10.8 Rastreabilidade com o guia

Seção do guia	Como este complemento estende
§4.1 (S4)	Adiciona consumo automático de `ods.s2.event` , `capture.command` , `clip.query` e a fonte de mídia em GCS no lugar do NVMe.
§4.2 (S5)	Troca o consumidor in-memory pelo Kafka real e liga a produção às métricas da 6c.
§4.3 (B6)	Liga OverlayPlayer ao S4 (descritor + URL assinada) e formaliza o artefato ZONES na saída do ZoneEditor.
§5 (DDD/Clean Architecture)	Novos adaptadores em `infrastructure/` (`gcs/` , `b3_bus/`) atrás dos contratos de domínio `IFileStorage` / `B3EventConsumer` ; domínio intocado.
§6 (Padrões GoF)	`GcsFileStorage` e `FileStorage` como Strategy de armazenamento; `KafkaEventTransport` mantém o papel Observer/Consumer.
§8 (Testes)	Fakes em memória para GCS e Kafka garantem testes unitários sem I/O; E2E do squad cobre a integração real.
§9 (Checklist)	Acrescentam-se: bootstrap configurável (sem endereço fixo), sem binário no barramento, idempotência por `message_id` e conformidade LGPD com `expires_at` refletido no bucket.